CONVOLUTIONAL NETWORKS

---

1. The Convolution Operation

Convolutional Neural Networks (CNNs) are a type of deep learning model mainly used for image processing, object detection, face recognition, medical imaging, video analysis, and many other AI applications.

The most important operation inside a CNN is called the **convolution operation**.

Basic Idea of Convolution

Imagine you are looking at a large image and trying to find a cat.

Instead of looking at the whole image at once, you look for small features like:

- Eyes
- Ears
- Nose
- Fur texture

CNNs work in the same way.

They use a small filter called a **kernel** or **filter** to scan different parts of the image and detect patterns.

How Convolution Works in CNN – Step by Step

Step 1: Input Image

The image is converted into numbers called **pixels**.

Example input image:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Here:

- Each number represents the brightness of a pixel.
- Larger numbers mean brighter pixels.

Step 2: Kernel / Filter

A small matrix called a **kernel** (or filter) is used to detect features.

Example kernel:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

This kernel checks diagonal patterns.

Step 3: Sliding the Kernel

The kernel moves across the image:

- Left → Right
- Top → Bottom

At every position:

1. Multiply matching values
2. Add all the results
3. Store the answer in a new matrix

This operation is called **Convolution**.

Full Calculation

Position 1

Kernel covers:

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$$

Multiply with kernel:

$$\begin{aligned} &(1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times 1) \\ &= 1 + 0 + 0 + 5 \\ &= 6 \end{aligned}$$

First output value = **6**

Position 2

Kernel moves right.

Covered area:

$$\begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$$

Calculation:

$$\begin{aligned} &(2 \times 1) + (3 \times 0) + (5 \times 0) + (6 \times 1) \\ &= 2 + 0 + 0 + 6 \\ &= 8 \end{aligned}$$

Second output value = **8**

Position 3

Kernel moves downward.

Covered area:

$$\begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}$$

Calculation:

$$\begin{aligned} &(4 \times 1) + (5 \times 0) + (7 \times 0) + (8 \times 1) \\ &= 4 + 0 + 0 + 8 \\ &= 12 \end{aligned}$$

Third output value = **12**

Position 4

Kernel moves right again.

Covered area:

$$\begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$$

Calculation:

$$(5 \times 1) + (6 \times 0) + (8 \times 0) + (9 \times 1)$$

$$= 5 + 0 + 0 + 9$$
$$= 14$$

Fourth output value = **14**

Final Output Matrix

After convolution, the new matrix becomes:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

Feature Map

The output produced after convolution is called a **feature map**.

The feature map highlights important features like:

- Edges
- Corners
- Shapes
- Textures

Different kernels detect different patterns.

For example:

- One kernel detects vertical edges
- Another detects horizontal edges
- Another detects curves

Feature Map

The output produced after convolution is called a **feature map**.

The feature map highlights important features like:

- Edges
- Corners
- Shapes
- Textures

Different kernels detect different patterns.

For example:

- One kernel detects vertical edges
- Another detects horizontal edges
- Another detects curves

Important Concepts

Discrete and Continuous Data

Computers work with discrete values (fixed pixels).

So CNNs mainly use discrete convolution instead of continuous mathematical convolution.

Cross-Correlation

In mathematics, convolution normally flips the kernel before applying it.

But in CNNs, most libraries skip the flipping step.

This operation is technically called **cross-correlation**, but people still commonly call it convolution.

Main Purpose of Convolution

The convolution operation helps the network:

- Detect local features
- Reduce computation
- Learn patterns automatically
- Preserve spatial relationships

This makes CNNs very powerful for image-related tasks.

2.1 Sparse Interactions

In traditional neural networks:

- Every input connects to every output.
- This creates too many connections.

For large images, this becomes very expensive.

CNN Solution

CNNs use only local connections.

A neuron looks only at a small region of the image called the:

Receptive Field

Example:

A neuron may look only at a 3×3 region instead of the whole image.

Advantages

1. Less Memory Usage

Fewer weights are needed.

2. Faster Computation

Less multiplication operations.

3. Better Learning

The network focuses on useful local patterns.

2.2 Parameter Sharing

CNNs use the same filter everywhere in the image.

Example:

If one filter can detect an eye in one position,
the same filter can detect an eye anywhere else.

This is called:

Parameter Sharing

Benefits

- Greatly reduces parameters
- Improves efficiency
- Makes training easier

- Helps recognize objects anywhere in the image

This is also called:

Tied Weights

2.3 Equivariant Representations

Convolution is translation equivariant.

This means:

If the object moves,
the detected feature also moves in the same way.

Example:

- Cat moves from left to right
- Detection map also shifts right

Importance

The network learns one feature detector that works everywhere.

This helps object recognition become more efficient.

Limitation

CNNs naturally handle:

- Translation

But they do not naturally handle:

- Rotation
- Scaling
- Large viewpoint changes

So techniques like:

- Data augmentation
- Rotated images
- Flipped images

are used during training.

Variable-Sized Inputs

CNNs can process images of different sizes because filters slide across the image.

This is a major advantage compared to traditional dense networks.

==========
==========
==========
==========

11 (a) Compare L1 and L2 Regularization with Examples

Introduction

Regularization is an important technique used in deep learning to reduce overfitting. Sometimes neural networks learn the training data too perfectly and fail to perform well on new data. Regularization helps the model generalize better by adding a penalty to the loss function. The two most common regularization techniques are L1 and L2 regularization.

L1 Regularization

L1 regularization adds the absolute values of weights to the loss function. It tries to reduce unnecessary weights and may even make some weights equal to zero.

Formula:

$$Loss = Original\ Loss + \lambda \sum | w |$$

Here:

- λ is the regularization parameter
- w represents weights

In L1 regularization, small weight values gradually become zero. Because of this, unnecessary features are removed automatically. Therefore, L1 regularization is also called a feature selection method.

For example, if the weights are:

w = [5, 0.2, 0.01, 7]

After applying L1 regularization:

w = [4.8, 0, 0, 6.9]

The smaller weights become zero.

Advantages of L1:

- Removes unnecessary features
- Produces sparse models
- Useful for large datasets with many features

Disadvantages:

- Training may become unstable
- Less smooth compared to L2

L2 Regularization

L2 regularization adds squared values of weights to the loss function. Instead of making weights zero, it reduces them gradually.

Formula:

$$Loss = Original\ Loss + \lambda \sum w^2$$

L2 regularization prevents weights from becoming very large. It produces smoother and more stable learning.

Example:

Before regularization:

w = [5, 0.2, 0.01, 7]

After L2 regularization:

w = [4.7, 0.1, 0.005, 6.5]

Weights become smaller but not exactly zero.

Advantages of L2:

- Stable learning
- Prevents overfitting effectively
- Most commonly used regularization method

Disadvantages:

- Does not completely remove features

Explain the Working of a CNN in Image Classification

Introduction

Convolutional Neural Networks (CNNs) are a type of deep learning model mainly used for image processing, object detection, face recognition, medical imaging, video analysis, and many other AI applications.

The most important operation inside a CNN is called the **convolution operatio**

Working of CNN

CNN works similarly to the human visual system. Initial layers detect simple features such as edges and corners, while deeper layers learn complex objects and patterns.

1. Input Layer

The image is given as input.

Example:
224 × 224 × 3 RGB image

---

2. Convolution Layer

Filters scan the image and extract features like:

- edges
- textures
- shapes

Feature maps are generated.

---

3. ReLU Activation Function

ReLU introduces non-linearity.

Formula:

$$f(x) = \max(0, x)$$

Negative values become zero and positive values remain unchanged.

If **x > 0**, the output is **x**

If **x ≤ 0**, the output is **0**

Advantages:

- Faster training
- Reduces vanishing gradient problem

---

4. Pooling Layer

Pooling is another important operation in CNNs.

Pooling reduces the size of feature maps while keeping important information.

Types:

1. Max Pooling
2. Average Pooling

Example:

1 5

2 3

Max pooling output = 5

Pooling reduces computation and overfitting.

---

5. Flattening

Feature maps are converted into a one-dimensional vector.

Example:

[[1,2], [3,4]] → [1,2,3,4]

---

6. Fully Connected Layer

The extracted features are combined and used for classification.

---

7. Output Layer

Final prediction is generated using Softmax activation.

Example:

- Cat
- Dog
- Car

CNN Flow

Input Image
→ Convolution
→ ReLU
→ Pooling
→ Flattening
→ Fully Connected Layer
→ Output

Conclusion

CNN extracts image features layer by layer and performs accurate image classification. It is widely used in modern computer vision systems.

13 (b) Explain Vanishing and Exploding Gradient Problems

Introduction

Vanishing gradient and exploding gradient are common problems that occur while training deep neural networks and Recurrent Neural Networks (RNNs). These problems happen during backpropagation when gradients are propagated through many layers.

Gradients are used to update weights during training. If gradients become too small or too large, the network cannot learn properly.

Vanishing Gradient Problem

The vanishing gradient problem occurs when gradient values become extremely small during backpropagation.

As gradients move backward through many layers, repeated multiplication of small values makes them close to zero. Because of this, earlier layers learn very slowly or stop learning completely.

This problem is common in:

- Deep neural networks
- Traditional RNNs

Effects of vanishing gradients:

- Slow training
- Poor learning in earlier layers
- Difficulty learning long-term dependencies
- Reduced model accuracy

Example:

If gradients are repeatedly multiplied:

$0.1 \times 0.1 \times 0.1 \times 0.1 = 0.0001$

The value becomes extremely small.

Exploding Gradient Problem

The exploding gradient problem occurs when gradients become extremely large during training.

During backpropagation, repeated multiplication of large values causes gradients to increase rapidly. This leads to unstable weight updates.

Effects of exploding gradients:

- Unstable training
- Very large weight values
- Model fails to converge
- Loss function becomes very high

Example:

$10 \times 10 \times 10 = 1000$

The gradient becomes extremely large.

Feature	**Vanishing Gradient**	**Exploding Gradient**
Gradient Value	Very small	Very large
Learning	Slow	Unstable

Feature	Vanishing Gradient	Exploding Gradient
Main Problem	No learning	Weight overflow
Common in	Deep RNNs	Deep networks
Solution	LSTM, ReLU	Gradient clipping

14 Illustrate Long Short-Term Memory (LSTM) Networks with Diagrams

Introduction

Long Short-Term Memory (LSTM) is a special type of Recurrent Neural Network (RNN). It is designed to overcome the limitations of traditional RNNs, especially the vanishing gradient problem.

LSTM can remember information for a long time using memory cells and gates. Because of this, it performs very well on sequential data such as text, speech, and time-series data.

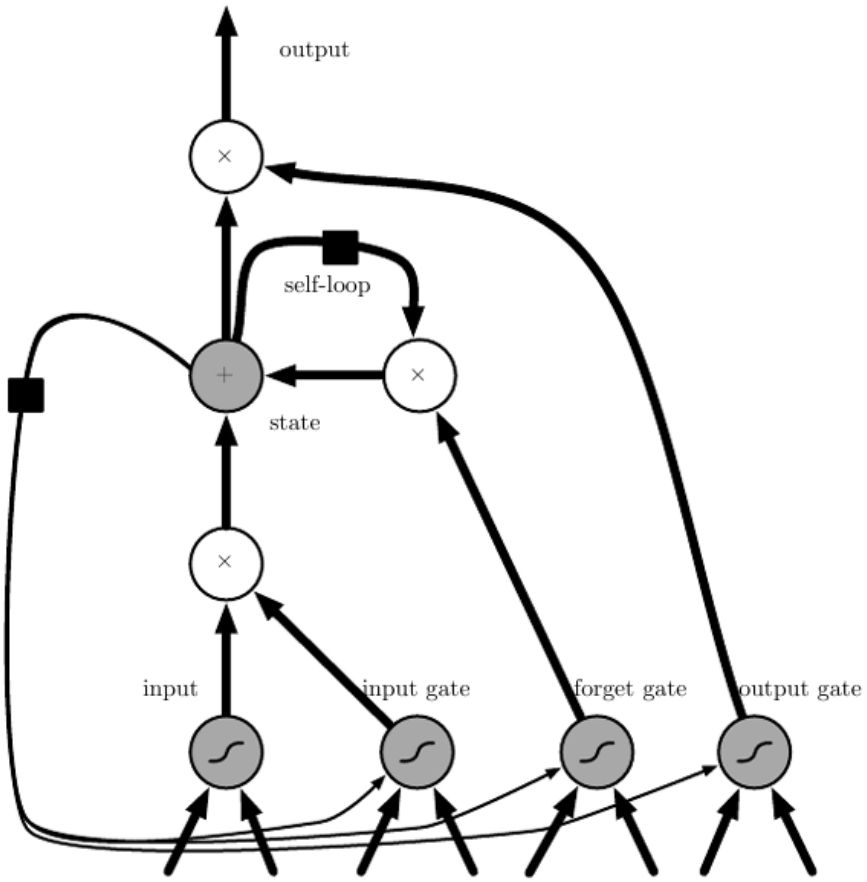
It is widely used in natural language processing and sequence modeling tasks.

Basic Components of LSTM

An LSTM network mainly contains:

1. Forget Gate
2. Input Gate
3. Cell State
4. Output Gate
5. Hidden State

These components work together to manage information flow.



1. Forget Gate

The forget gate decides which information should be removed from memory.

Formula:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

If the output is close to:

- 0 → information is forgotten
- 1 → information is retained

---

2. Input Gate

The input gate decides which new information should be stored in memory.

Formula:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate memory values:

$$\widetilde{C}_t = tanh(W_c[h_{t-1}, x_t] + b_c)$$

---

3. Cell State Update

The old memory and new information are combined to update the cell state.

Formula:

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t$$

The cell state acts as long-term memory.

---

4. Output Gate

The output gate determines the final output of the LSTM cell.

Formula:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden state:

$$h_t = o_t * tanh(C_t)$$

Working of LSTM

LSTM works step-by-step:

1. Forget unnecessary information
2. Store useful information
3. Update memory cell
4. Generate output
5. Pass memory to next time step

This controlled information flow helps LSTM remember important patterns for long durations.

Advantages of LSTM

1. Solves vanishing gradient problem
2. Learns long-term dependencies

3. Better memory management
4. High accuracy in sequential tasks
5. Stable training

Applications of LSTM

- Speech recognition
- Language translation
- Text prediction
- Chatbots
- Time-series forecasting
- Sentiment analysis

15 (a) Describe Backpropagation Through Time (BPTT)

Introduction

Backpropagation Through Time (BPTT) is the training algorithm used for Recurrent Neural Networks (RNNs). It is an extension of normal backpropagation designed specifically for sequential data.

BPTT helps the RNN learn patterns and dependencies in sequential data such as text, speech, and time series.

Working of BPTT

BPTT trains RNNs in four major steps.

Step 1: Forward Pass

The input sequence is processed one step at a time.

At each time step:

- hidden state is updated
- output is generated
- Example:
- $x1 \rightarrow x2 \rightarrow x3 \rightarrow x4$

Step 2: Compute Loss

The error at each time step is calculated.

The losses from all time steps are combined.

$$L = \sum_{t=1}^{T} L_t$$

Step 3: Unfold the RNN

The recurrent network is expanded across time steps.

This converts the RNN into a deep feedforward-like structure.

Example:

h1 → h2 → h3 → h4

Step 4: Backward Pass

Gradients are propagated backward through all time steps.

Weights are updated using gradient descent to reduce error.

Problems in BPTT

Vanishing Gradient

Gradients become extremely small and long-term learning becomes difficult.

Exploding Gradient

Gradients become very large and training becomes unstable.

Solutions

1. LSTM
2. GRU
3. Gradient clipping
4. Proper initialization

Applications

- Speech recognition
- Language modeling

- Text generation
- Machine translation
- Time-series prediction

15 (b) Explain the Concept of Recursive Neural Networks

Introduction

Recursive Neural Networks are neural networks designed to process hierarchical or tree-structured data.

Unlike RNNs that process sequences linearly, Recursive Neural Networks combine smaller structures recursively to form larger representations.

They are mainly used in natural language processing and structured data analysis.

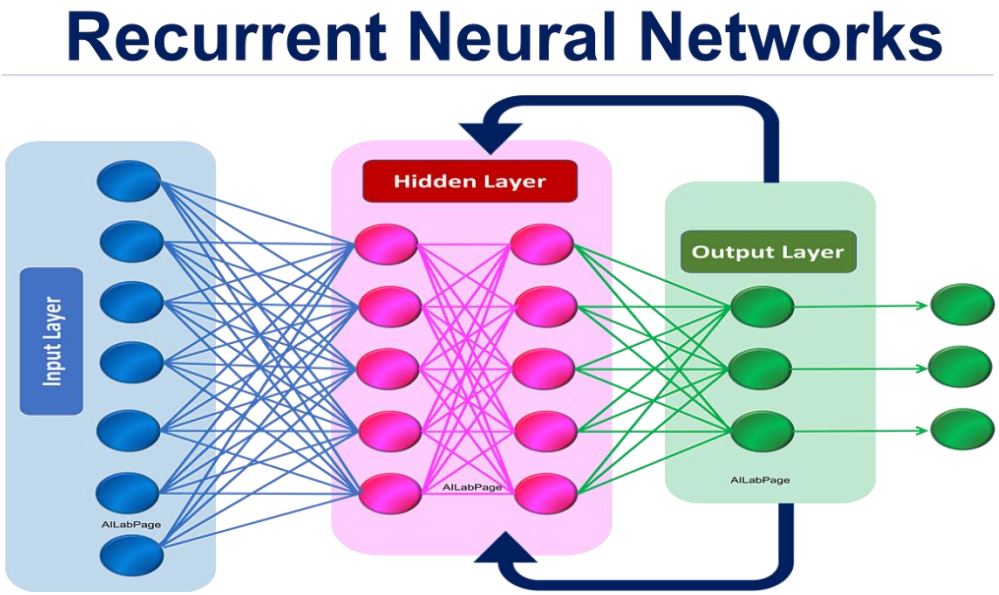
Working Principle

Recursive Neural Networks repeatedly apply the same weights on structured input data.

Steps:

1. Child nodes are processed first
2. Outputs are combined
3. Parent node is generated
4. Process continues recursively
5. Final root representation is obtained

Structure



Child Node 1 + Child Node 2
↓
Parent Node
↓
Root Node

This hierarchical processing helps understand structural relationships.

Example

Sentence:

“Deep learning is powerful”

Words are combined step-by-step to understand the meaning of the complete sentence.

Advantages

1. Handles hierarchical data effectively
2. Captures structural relationships
3. Reuses parameters efficiently
4. Good for tree-based structures

Limitations

1. Complex training process

2. High computational cost
3. Difficult implementation

Limitations of RNN

4. Suffers from vanishing gradient problem.
5. Suffers from exploding gradient problem.
6. Cannot remember long-term dependencies effectively.
7. Training process is slow because data is processed sequentially.
8. Parallel computation is difficult.
9. Requires high computational power for long sequences.
10. Basic RNN has short memory.
11. Optimization becomes difficult in deep networks.
12. Training may become unstable.
13. Performance decreases for very large datasets and long sequences.

Applications

- Natural language processing
- Syntax tree analysis
- Sentiment analysis
- Scene understanding

Describe Gated Recurrent Unit (GRU) and Compare it with LSTM

Introduction

Gated Recurrent Unit (GRU) is a special type of Recurrent Neural Network designed to solve the vanishing gradient problem. GRU is similar to LSTM but has a simpler architecture.

It uses gates to control information flow and helps the network remember long-term dependencies.

Components of GRU

GRU mainly contains:

1. Update Gate
2. Reset Gate

Unlike LSTM, GRU does not have a separate cell state.

1. Update Gate

The update gate decides how much previous information should be retained.

Formula:

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

This gate controls memory updating.

---

2. Reset Gate

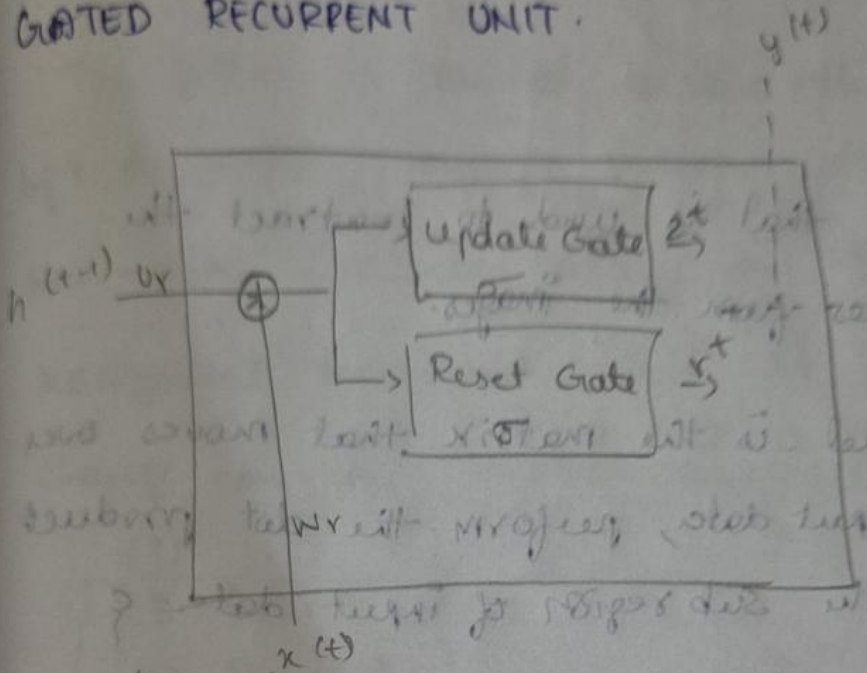
The reset gate decides how much old information should be forgotten.

Formula:

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

This helps remove unnecessary information.

GATED RECURRENT UNIT.



Only 2 gate

i) update gate - how much of past memory to retain

ii) Reset gate.

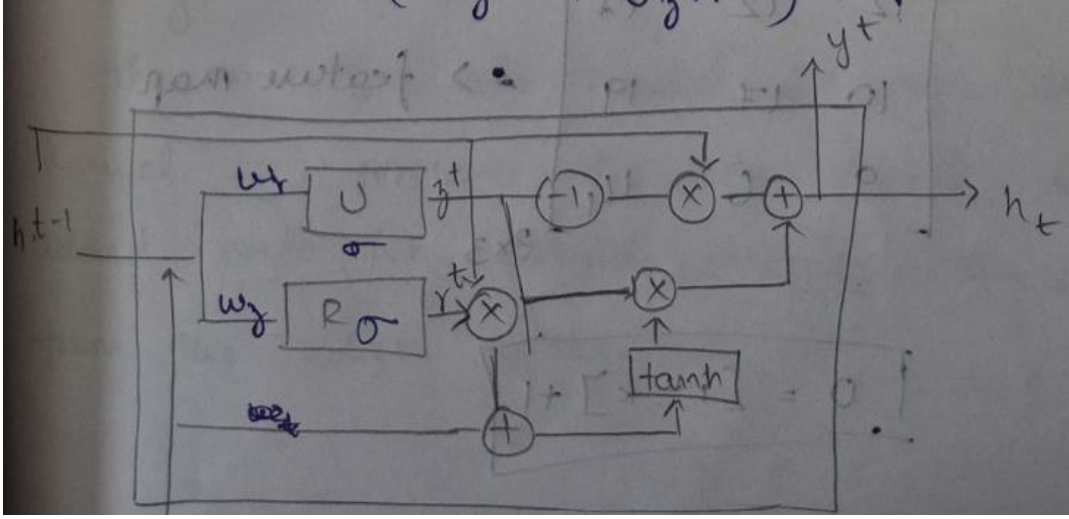
i) update gate - how much of past memory to retain

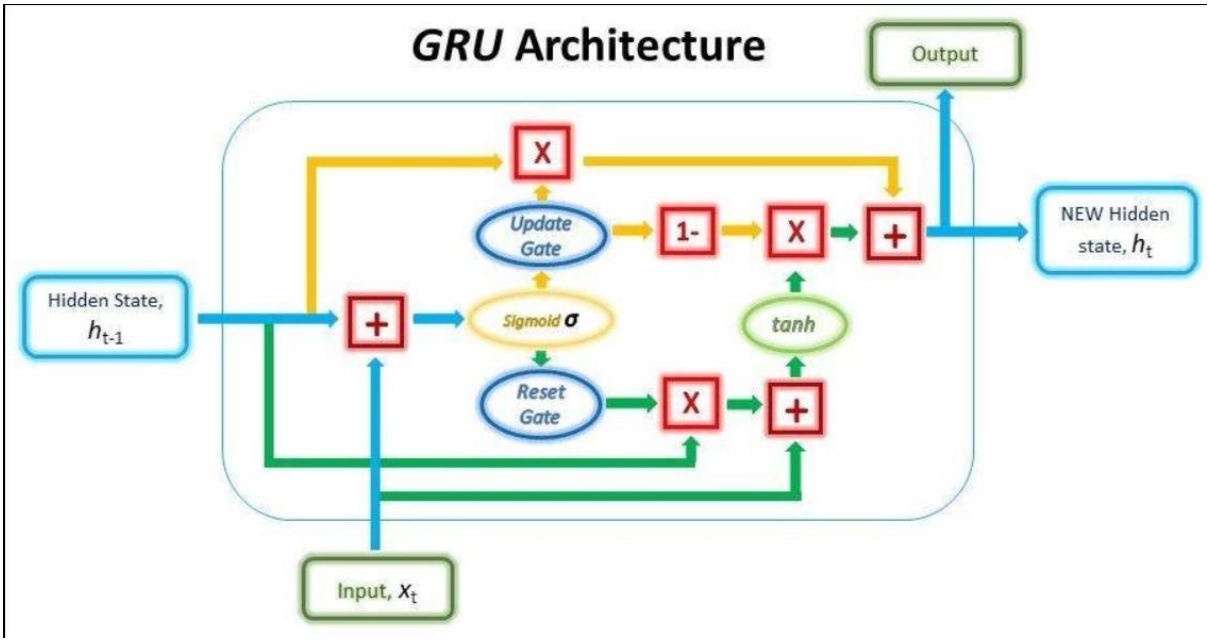
ii) Reset gate

how much of past memory to forget

$$r^t = \sigma(w_r x^t + U_r h^{t-1}) \rightarrow \text{reset}$$

$$z^t = \sigma(w_z x^t + U_z h^{t-1}) \rightarrow \text{update}$$





17 (b) Summarize how Noise Injection can be used as a Regularization Method

Introduction

Noise injection is a regularization technique where random noise is added during training. This prevents the model from memorizing exact training patterns and improves robustness.

The model learns to handle variations in input data and generalizes better.

Working of Noise Injection

Noise can be added to:

1. Input data
2. Hidden layers
3. Weights

This randomness forces the network to learn stable patterns instead of exact values.

Types of Noise Injection

1. Input Noise

Random noise is added to training inputs.

Example:
Small distortions added to images.

This improves robustness.

---

2. Weight Noise

Noise is added to network weights during training.

This prevents dependency on exact weight values.

---

3. Activation Noise

Noise is added to hidden layer activations.

This improves generalization.

Comparison Between Sobel and Prewitt Edge Operators

Feature	Sobel Operator	Prewitt Operator
Definition	Edge detection operator that gives more importance to central pixels	Edge detection operator that uses equal weights
Kernel Values	Uses weighted coefficients (2 in center)	Uses uniform coefficients
Noise Handling	Better noise suppression	More sensitive to noise
Edge Detection Accuracy	More accurate	Less accurate
Computational Complexity	Slightly higher	Simpler and faster
Gradient Calculation	Provides smoother gradient	Provides rough gradient
Image Quality	Produces clearer edges	Produces less sharp edges
Usage	Commonly used in image processing	Used for simple edge detection tasks

Gradient-Based Edge Detection

Introduction

Gradient-based edge detection is a technique used in image processing to identify edges in an image. Edges represent sudden changes in pixel intensity and help identify object boundaries, shapes, and important features.

This method calculates the intensity change between neighboring pixels. Large intensity changes indicate the presence of edges.

Working Principle

In gradient-based edge detection, the image gradient is calculated in horizontal and vertical directions.

The gradient measures how quickly pixel values change.

- Small gradient → smooth region
- Large gradient → edge region

The two important gradient directions are:

1. Horizontal Gradient (Gx)
2. Vertical Gradient (Gy)

Gradient Magnitude

The strength of the edge is calculated using:

$$G = \sqrt{G_x^2 + G_y^2}$$

Where:

- G = gradient magnitude
- Gx = horizontal gradient
- Gy = vertical gradient

A higher gradient magnitude indicates a stronger edge.

Gradient Direction

Edge direction is calculated using:

$$\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$$

This gives the direction of the edge.

Steps in Gradient-Based Edge Detection

1. Read the input image.
2. Apply gradient operators.
3. Calculate horizontal and vertical gradients.
4. Compute gradient magnitude.
5. Detect pixels with high gradient values as edges.

Common Gradient Operators

1. Sobel Operator

Uses weighted kernels and provides better noise suppression.

2. Prewitt Operator

Uses simple kernels for edge detection.

3. Roberts Operator

Uses small 2×2 kernels for fast edge detection.

Comparison Between Linear and Non-Linear Filters

Feature	Linear Filter	Non-Linear Filter
Definition	Output is calculated using linear operations such as addition and averaging	Output is calculated using non-linear operations
Working Principle	Uses weighted sum of neighboring pixels	Uses operations like median, maximum, or minimum
Noise Removal	Effective for Gaussian noise	Effective for salt-and-pepper noise
Edge Preservation	Blurs edges	Preserves edges better
Computational Complexity	Simple and faster	More complex
Image Smoothing	Produces smoother images	Maintains image details
Mathematical Property	Follows superposition principle	Does not follow superposition principle
Examples	Mean filter, Gaussian filter	Median filter, Max filter